

Recursive File Explorer

Developer Notes

Project: file-explorer-react-question-01

Stack: React 18, Create React App (react-scripts 5)

Purpose: Machine-coding interview practice - build a nested file/folder tree UI with add operations.

1. What This Project Teaches

This project is a minimal but complete React application that demonstrates how to model hierarchical data (a file system tree), render it recursively, manage shared state at the top level, and mutate nested immutable data structures using a custom hook. It is intentionally small so students can focus on core React patterns rather than routing, API calls, or styling frameworks.

- Component composition and props drilling (callback pattern)
- useState for local UI state and application-level tree state
- Recursive rendering - a component rendering itself for child nodes
- Custom hooks to separate algorithmic logic from UI
- Immutable tree updates via spread operator during recursion
- Event handling: click, keyboard (Enter), blur, stopPropagation

2. High-Level Architecture

The app follows a unidirectional data flow. App.js owns the tree. Folder.js displays nodes and collects user input. useTraverseTree.js performs the insert algorithm. folderData.js provides initial seed data.

```
public/index.html
|
src/index.js  (ReactDOM.render)
|
src/App.js    (state: explorerData)
|
src/components/Folder.js  (recursive UI)
|
src/hooks/use-traverse-tree.js  (insertNode)
|
src/data/folderData.js  (initial tree)
```

3. Project File Structure

3.1 public/index.html

Standard Create React App HTML shell. Contains a single `<div id="root"></div>`. React replaces this empty div with the rendered component tree when index.js runs. No application logic lives here.

3.2 src/index.js - Application Bootstrap

Entry point. Imports React, ReactDOM, and App. Calls ReactDOM.render to mount `<App />` inside StrictMode. StrictMode helps surface unsafe lifecycles and unexpected side effects during development by intentionally double-rendering in dev mode.

```
ReactDOM.render(  
  <StrictMode><App /></StrictMode>,  
  document.getElementById('root')  
)
```

3.3 src/data/folderData.js - Seed Tree Data

Exports a default JavaScript object representing the root of the explorer. Every node uses the same schema, which keeps recursion simple.

```
{  
  id: string,          // unique key for React lists and insert targeting  
  name: string,        // label shown in UI  
  isFolder: boolean,   // branch vs leaf  
  items: array         // children (empty for files)  
}
```

The sample tree mimics a real project: root > public/, src/, package.json. Nested folders like public/public nested 1/ demonstrate multi-level recursion in the UI.

3.4 src/App.js - Root Component & State Owner

App is the single source of truth for the entire tree.

- useState(explorer) - explorerData holds the current tree
- useTraverseTree() - returns insertNode (and stub deleteNode/renameNode)
- handleInsertNode(folderId, item, isFolder) - calls insertNode, then setExplorerData
- Renders one <Folder /> at the root, passing explorerData and the callback

When any nested Folder adds an item, the callback bubbles up to App without prop drilling the setter itself - only the stable handleInsertNode function is passed down recursively.

3.5 src/hooks/use-traverse-tree.js - Tree Mutation Logic

Custom hook encapsulating recursive tree algorithms. Components stay declarative; the hook stays imperative/algorithmic.

insertNode(tree, folderId, item, isFolder) algorithm:

- Base case: if tree.id === folderId and tree.isFolder, unshift new node into tree.items
- New node id: new Date().getTime() (timestamp as unique string-ish id)
- Recursive case: map over tree.items, call insertNode on each child
- Return { ...tree, items: latestNode } to produce updated references up the ancestry chain

Important nuance: at the matched folder, the function mutates tree.items in place via unshift, then returns the same tree reference. For ancestors, it returns shallow copies. In practice React still re-renders because setExplorerData receives a new root object when the match is not at root. Students extending this project should prefer fully immutable updates for correctness.

deleteNode and renameNode are empty stubs - intended student exercises.

3.6 src/components/Folder.js - Recursive UI Component

The heart of the UI. Folder receives explorer (one node) and handleInsertNode. It decides rendering based on explorer.isFolder.

Local state:

- expand (boolean) - toggles visibility of children

- showInput ({ visible, isFolder }) - controls inline add input

Folder branch UI:

- Click row toggles expand/collapse
- 'Folder +' / 'File +' buttons call handleNewFolder with stopPropagation so the row does not toggle
- When adding, parent auto-expands and shows a text input with autoFocus
- Enter (keyCode 13) with non-empty value calls handleInsertNode(explorer.id, value, isFolder)
- onBlur hides input without saving
- explorer.items.map renders child <Folder key={exp.id} ... /> - recursion

File branch UI: simple span with file icon and name - leaf node, no children.

3.7 src/styles.css - Presentation

Plain CSS (no CSS modules or styled-components). Classes: .folder (row layout, gray background), .file (indented leaf), .inputContainer and .inputContainer__input (inline add field). Inline styles in Folder.js handle dynamic paddingLeft and display for expand/collapse.

4. End-to-End Flow: Adding a File

Example: user expands src/, clicks 'File +', types 'utils.js', presses Enter.

- 1. handleNewFolder sets expand=true, showInput={ visible:true, isFolder:false }
- 2. Input renders; user types and presses Enter
- 3. onAddFolder calls handleInsertNode('7', 'utils.js', false) - '7' is src folder id
- 4. App.handleInsertNode runs insertNode(explorerData, '7', 'utils.js', false)
- 5. insertNode recurses until tree.id === '7', prepends new node to src.items
- 6. setExplorerData(finalTree) triggers App re-render
- 7. Folder tree re-renders; src shows new file at top of its children list

5. Key React Concepts for Students

5.1 Recursion in JSX

Folder renders Folder for each child. React reconciles using key={exp.id}. Without stable keys, sibling reordering or insertion can cause incorrect DOM reuse.

5.2 Lifting State Up

Tree data lives in App because multiple Folder instances need one shared tree. Local UI state (expanded, input visible) stays in each Folder - it does not need to be global.

5.3 Custom Hooks

useTraverseTree has no JSX. It returns functions. This separation makes the insert algorithm testable and reusable without coupling to the Folder component.

5.4 Controlled vs Local State

The add input is uncontrolled (uses e.target.value on keydown, not useState per keystroke). That is acceptable for a simple interview demo; a production app might use controlled inputs.

6. Running the Project

```
npm install
npm start    # dev server at http://localhost:3000
npm test     # Jest + React Testing Library
npm run build
```

7. Suggested Extensions (Learning Exercises)

- Implement deleteNode in use-traverse-tree.js and wire a delete button on folders/files
- Implement renameNode with inline editing on double-click
- Persist tree to localStorage on each update
- Replace keyCode with e.key === 'Enter' (keyCode is legacy)
- Make insertNode fully immutable (no in-place unshift on matched node)
- Add PropTypes or TypeScript for the node schema
- Write tests for insertNode covering root, nested, and not-found cases

8. Common Interview Discussion Points

- Time complexity of insertNode: $O(n)$ where n is number of nodes visited in worst case
- Why recursion fits trees: each node has the same shape as the root
- Trade-off: prop drilling vs Context API for handleInsertNode at scale
- Accessibility gaps in current UI: buttons in clickable div, no aria-expanded - good improvement topic

9. Quick Reference - Node Schema & API

```
Node: { id, name, isFolder, items[] }

handleInsertNode(folderId, name, isFolder)
insertNode(tree, folderId, item, isFolder) => updatedTree
deleteNode() - TODO
renameNode() - TODO
```